

PHASE 2 PROMPT — DATABASE DESIGN AND DOMAIN MODEL FOR TOURISTSYSTEM

Ты — Senior Database Architect и Domain-Driven Design эксперт с большим опытом проектирования production-ready систем.

Проект: TouristSystem
Тип проекта: Tourist Booking System
Backend: C# / ASP.NET Core .NET 9
Database: PostgreSQL
Frontend: React + TypeScript
Architecture: Clean Architecture

Твоя задача в этой фазе — полностью спроектировать базу данных и domain model для проекта TouristSystem.

ВАЖНЫЕ ПРАВИЛА:

1. Не пиши backend code.
2. Не пиши frontend code.
3. Не переходи к API и controllers.
4. Работай только над database design и domain model.
5. Объясняй всё просто, но профессионально.
6. Используй PostgreSQL best practices.
7. Используй UUID для primary keys.
8. Продумай связи, индексы, constraints, enums и audit fields.
9. Проект должен быть готов к масштабированию.
10. Не делай слабую базу данных.
11. Не пропускай мелкие детали.
12. Если есть несколько вариантов связи, выбери лучший и объясни почему.
13. В конце дай final database checklist.
14. Не пиши код, пока я отдельно не скажу "give SQL" или "give entity code".

===== 1. PHASE GOAL
=====

Объясни цель этой фазы.

Нужно спроектировать базу данных так, чтобы она поддерживала:

- users
- roles
- authentication
- refresh tokens
- tourist places
- hotels
- restaurants
- transports
- guides
- bookings
- reviews

- favorites
- notifications
- audit logs
- admin management
- owner management
- soft delete
- filtering
- searching
- sorting
- reporting
- future payment integration

Объясни, что хорошая база данных — это фундамент проекта, и если здесь сделать ошибку, потом backend и frontend будут сложными.

===== 2. DATABASE PRINCIPLES =====

Опиши database principles для проекта:

- PostgreSQL as main relational database
- UUID primary keys
- normalized tables
- clear foreign keys
- meaningful indexes
- check constraints
- unique constraints
- audit fields
- soft delete
- timestamps in UTC
- safe decimal types for money
- no duplicated important data
- scalable structure
- role-based ownership
- future payment support

Объясни, почему нельзя хранить всё в одной большой таблице.

===== 3. MAIN ENTITIES =====

Определи главные domain entities:

1. User
2. RefreshToken
3. Place
4. Hotel
5. Restaurant
6. Transport
7. Guide
8. Booking

- 9. Review
- 10. Favorite
- 11. Notification
- 12. AuditLog

Для каждой entity объясни:

- что она означает
- зачем она нужна
- кто её создаёт
- кто её может изменять
- какие связи имеет с другими entity

===== 4. USERS TABLE DESIGN
=====

Спроектируй Users table.

Обязательно включи:

- Id
- FullName
- Email
- PasswordHash
- PhoneNumber
- Role
- IsActive
- EmailConfirmed
- CreatedAt
- UpdatedAt
- LastLoginAt
- IsDeleted

Объясни:

- почему Email должен быть unique
- почему PasswordHash не должен хранить plain password
- почему Role важен
- почему IsActive и IsDeleted разные вещи
- какие индексы нужны
- какие validation rules нужны

Определи роли:

- Tourist
- Admin
- SuperAdmin
- Guide
- HotelOwner
- RestaurantOwner
- TransportOwner

===== 5. REFRESH TOKENS TABLE DESIGN
=====

Спроектируй RefreshTokens table.

Обязательно включи:

- Id
- UserId
- TokenHash
- ExpiresAt
- CreatedAt
- RevokedAt
- ReplacedByTokenId
- DeviceInfo
- IPAddress
- IsRevoked

Объясни:

- зачем нужен refresh token
- почему token лучше хранить как hash
- как logout должен revoke token
- как refresh token rotation работает
- какие индексы нужны

===== 6. PLACES TABLE DESIGN
=====

Спроектируй Places table.

Обязательно включи:

- Id
- Name
- Slug
- Description
- City
- Address
- Latitude
- Longitude
- ImageUrl
- EntryFee
- RatingAverage
- ReviewsCount
- Status
- CreatedByUserId
- CreatedAt
- UpdatedAt
- IsDeleted

Объясни:

- зачем нужен Slug
- зачем RatingAverage и ReviewsCount
- почему City должен индексироваться
- как place будет использоваться в search/filter
- кто может создавать place
- кто может менять place

===== 7. HOTELS TABLE DESIGN
=====

Спроектируй Hotels table.

Обязательно включи:

- Id
- OwnerId
- Name
- Slug
- Description
- City
- Address
- Latitude
- Longitude
- PhoneNumber
- WebsiteUrl
- ImageUrl
- PricePerNight
- Stars
- RatingAverage
- ReviewsCount
- TotalRooms
- AvailableRooms
- HasWifi
- HasParking
- HasPool
- HasGym
- HasRestaurant
- IsFamilyFriendly
- IsLuxury
- IsBudget
- Status
- CreatedAt
- UpdatedAt
- IsDeleted

Объясни:

- почему OwnerId нужен
- почему PricePerNight должен быть decimal

- почему Stars должен быть 1–5
- почему amenities лучше хранить отдельными boolean fields на первом этапе
- какие фильтры frontend сможет делать
- какие индексы нужны

===== 8. RESTAURANTS TABLE DESIGN =====

Спроектируй Restaurants table.

Обязательно включи:

- Id
- OwnerId
- Name
- Slug
- Description
- City
- Address
- Latitude
- Longitude
- PhoneNumber
- WebsiteUrl
- ImageUrl
- CuisineType
- PriceRange
- OpeningHours
- RatingAverage
- ReviewsCount
- HasDelivery
- HasWifi
- HasParking
- Status
- CreatedAt
- UpdatedAt
- IsDeleted

Объясни:

- зачем CuisineType
- зачем PriceRange
- как фильтровать рестораны
- какие индексы нужны
- кто может управлять рестораном

===== 9. TRANSPORTS TABLE DESIGN =====

Спроектируй Transports table.

Обязательно включи:

- Id
- OwnerId
- Name
- Type
- OriginCity
- DestinationCity
- DepartureTime
- ArrivalTime
- PricePerSeat
- TotalSeats
- AvailableSeats
- VehicleNumber
- ContactPhone
- Status
- CreatedAt
- UpdatedAt
- IsDeleted

TransportType enum:

- Bus
- Taxi
- Car
- Train
- Van

Объясни:

- почему OriginCity и DestinationCity индексируются
- почему AvailableSeats не должен быть меньше 0
- почему ArrivalTime должен быть после DepartureTime
- как transport booking будет работать

===== 10. GUIDES TABLE DESIGN
=====

Спроектируй Guides table.

Обязательно включи:

- Id
- UserId
- Bio
- Languages
- City
- PricePerDay
- ExperienceYears
- RatingAverage
- ReviewsCount
- ImageUrl

- IsAvailable
- Status
- CreatedAt
- UpdatedAt
- IsDeleted

Объясни:

- почему Guide связан с User
- почему UserId должен быть unique
- как хранить Languages на первом этапе
- как фильтровать гидов по языку, городу, цене и рейтингу
- какие индексы нужны

===== 11. BOOKINGS TABLE DESIGN
=====

Спроектируй Bookings table.

Обязательно включи:

- Id
- UserId
- BookingType
- ReferenceId
- StartDate
- EndDate
- GuestsCount
- Quantity
- TotalPrice
- Status
- Notes
- CreatedAt
- UpdatedAt
- CancelledAt
- ConfirmedAt
- IsDeleted

BookingType enum:

- Hotel
- Transport
- Guide

BookingStatus enum:

- Pending
- Confirmed
- Cancelled
- Expired
- Completed

Объясни:

- почему используется BookingType + ReferenceId
- какие плюсы и минусы polymorphic reference
- как проверять ReferenceId на уровне service
- почему TotalPrice хранится в booking
- почему status важен
- какие индексы нужны
- какие бизнес-правила нужны

===== 12. REVIEWS TABLE DESIGN
=====

Спроектируй Reviews table.

Обязательно включи:

- Id
- UserId
- ReviewType
- ReferenceId
- Rating
- Comment
- Status
- CreatedAt
- UpdatedAt
- IsDeleted

ReviewType enum:

- Place
- Hotel
- Guide
- Restaurant

Объясни:

- почему Rating должен быть 1-5
- почему один user не должен оставлять много reviews на один объект
- как обновлять RatingAverage и ReviewsCount
- почему reviews нужны moderation
- какие индексы нужны

===== 13. FAVORITES TABLE DESIGN
=====

Спроектируй Favorites table.

Обязательно включи:

- Id
- UserId

- FavoriteType
- ReferenceId
- CreatedAt

FavoriteType enum:

- Place
- Hotel
- Restaurant
- Guide

Объясни:

- зачем favorites
- как не допустить duplicate favorite
- какие индексы нужны
- как frontend будет показывать saved items

===== 14. NOTIFICATIONS TABLE DESIGN
=====

Спроектируй Notifications table.

Обязательно включи:

- Id
- UserId
- Title
- Message
- Type
- IsRead
- CreatedAt
- ReadAt

NotificationType enum:

- Booking
- Review
- System
- Admin
- Payment

Объясни:

- зачем notifications
- какие события создают notifications
- как пользователь будет видеть unread count
- какие индексы нужны

===== 15. AUDIT LOGS TABLE DESIGN
=====

Спроектируй AuditLogs table.

Обязательно включи:

- Id
- UserId
- Action
- EntityName
- EntityId
- OldValues
- NewValues
- IPAddress
- UserAgent
- CreatedAt

Объясни:

- зачем audit logs
- почему admin actions должны логироваться
- почему OldValues/NewValues можно хранить как JSONB
- какие индексы нужны
- какие действия обязательно логировать

===== 16. ENUMS DESIGN
=====

Определи все enums проекта:

- UserRole
- BookingType
- BookingStatus
- TransportType
- ReviewType
- FavoriteType
- NotificationType
- EntityStatus
- PriceRange

Для каждого enum объясни:

- зачем он нужен
- где используется
- какие значения содержит

===== 17. RELATIONSHIPS
=====

Опиши все relationships:

- User one-to-many RefreshTokens
- User one-to-many Bookings
- User one-to-many Reviews

- User one-to-many Favorites
- User one-to-many Notifications
- User one-to-one Guide
- User one-to-many Hotels as Owner
- User one-to-many Restaurants as Owner
- User one-to-many Transports as Owner
- Booking polymorphic relation to Hotel / Transport / Guide
- Review polymorphic relation to Place / Hotel / Guide / Restaurant
- Favorite polymorphic relation to Place / Hotel / Restaurant / Guide

Объясни:

- какие связи будут enforced by database
- какие связи будут enforced by application service
- почему polymorphic relation требует аккуратности

===== 18. INDEX STRATEGY
=====

Определи index strategy.

Нужно описать индексы для:

- Users.Email
- Places.City
- Places.Slug
- Hotels.City
- Hotels.PricePerNight
- Hotels.Stars
- Hotels.RatingAverage
- Restaurants.City
- Restaurants.CuisineType
- Transports.OriginCity
- Transports.DestinationCity
- Transports.DepartureTime
- Guides.City
- Guides.PricePerDay
- Bookings.UserId
- Bookings.Status
- Bookings.BookingType + ReferenceId
- Reviews.ReviewType + ReferenceId
- Favorites.UserId + FavoriteType + ReferenceId
- Notifications.UserId + IsRead
- AuditLogs.EntityName + EntityId

Объясни, какие индексы нужны для search, filter, sort и admin dashboard.

===== 19. CONSTRAINT STRATEGY
=====

Определи constraints:

- required fields
- unique constraints
- foreign keys
- check constraints
- decimal precision
- max string lengths
- valid date ranges
- rating 1–5
- stars 1–5
- price ≥ 0
- seats ≥ 0
- end date after start date
- arrival time after departure time

Объясни, какие constraints лучше держать в database, а какие в application validation.

===== 20. SOFT DELETE STRATEGY
=====

Объясни soft delete strategy.

Нужно включить:

- IsDeleted
- DeletedAt
- DeletedByUserId
- global query filter in EF Core
- admin restore option
- why not physically delete important records
- exceptions where hard delete may be allowed

===== 21. AUDIT FIELDS STRATEGY
=====

Определи common audit fields:

- CreatedAt
- CreatedByUserId
- UpdatedAt
- UpdatedByUserId
- DeletedAt
- DeletedByUserId
- IsDeleted

Объясни:

- где эти поля нужны
- где не нужны
- как backend должен заполнять их автоматически

- почему timestamps должны быть UTC

===== 22. MONEY AND PRICE RULES =====

Опиши правила для money fields:

- decimal instead of double
- precision 10,2 or 12,2
- no negative price
- TotalPrice should be snapshot
- price changes should not affect old bookings

Примеры money fields:

- Hotel.PricePerNight
- Guide.PricePerDay
- Transport.PricePerSeat
- Place.EntryFee
- Booking.TotalPrice

===== 23. SEARCH AND FILTER SUPPORT =====

Объясни, как database будет поддерживать frontend filters:

Hotels:

- city
- price range
- stars
- rating
- available rooms
- pool
- gym
- restaurant
- family
- luxury
- budget

Places:

- city
- rating
- entry fee
- name search

Restaurants:

- city
- cuisine
- price range

- rating

Transports:

- origin
- destination
- departure date
- price
- available seats

Guides:

- city
- languages
- price
- rating
- availability

===== 24. ERD TEXT DIAGRAM
=====

Дай ERD в текстовом виде.

Покажи главные связи между:

- Users
- RefreshTokens
- Places
- Hotels
- Restaurants
- Transports
- Guides
- Bookings
- Reviews
- Favorites
- Notifications
- AuditLogs

ERD должен быть понятным для junior developer.

===== 25. DOMAIN MODEL RULES
=====

Опиши domain model rules:

- Entity base class
- Auditable entity
- Soft deletable entity
- aggregate roots
- value objects where useful
- enums in Domain layer
- no EF Core attributes inside Domain if using Fluent API

- business rules should not live in controllers

Объясни, какие entities будут aggregate roots.

===== 26. FINAL OUTPUT FORMAT
=====

Ответ должен быть структурирован так:

1. Phase 2 Goal
2. Database Principles
3. Full Tables Design
4. Enums
5. Relationships
6. Index Strategy
7. Constraint Strategy
8. Soft Delete Strategy
9. Audit Strategy
10. Search and Filter Support
11. ERD Text Diagram
12. Domain Model Rules
13. Final Checklist

Не пиши код.

Начни Phase 2.